

گزارش یادگیری هفته دوم: دیتابیس‌ها، SQL، کش و بهینه‌سازی

۱. مقدمه‌ای بر دیتابیس‌ها

دیتابیس یا پایگاه داده، مجموعه‌ای سازمان‌یافته از اطلاعات است که به صورت الکترونیکی ذخیره و مدیریت می‌شود. هدف اصلی آن، ذخیره‌سازی پایدار، بازیابی سریع و مدیریت کارآمد داده‌هاست.

دو دسته اصلی دیتابیس‌ها عبارتند از:

- دیتابیس‌های رابطه‌ای (Relational): داده‌ها در قالب جدول‌هایی با سطر و ستون ذخیره می‌شوند. ارتباط بین جدول‌ها با کلید خارجی تعریف می‌شود. PostgreSQL، MySQL و SQLite از معروف‌ترین آن‌ها هستند.
- دیتابیس‌های غیررابطه‌ای (NoSQL): ساختار منعطف‌تری دارند و انواع مختلفی مانند سندی (MongoDB)، کلید-مقدار (Redis) و گرافی را شامل می‌شوند.

۲. دیتابیس‌های رابطه‌ای و اصول ACID

دیتابیس‌های رابطه‌ای برای حفظ یکپارچگی داده‌ها از چهار اصل اساسی پیروی می‌کنند:

- تجزیه‌ناپذیری (Atomicity): هر تراکنش یا به طور کامل انجام می‌شود یا اصلاً انجام نمی‌شود.
- یکپارچگی (Consistency): تراکنش، دیتابیس را از یک حالت معتبر به حالت معتبر دیگر منتقل می‌کند.
- جداسازی (Isolation): تراکنش‌های هم‌زمان بر یکدیگر تأثیر نمی‌گذارند.
- ماندگاری (Durability): پس از ثبت تراکنش، داده حتی در صورت قطع برق حفظ می‌شود.

این اصول باعث می‌شود دیتابیس‌های رابطه‌ای برای کاربردهای حساس مانند سیستم‌های مالی و بانکی انتخابی ایده‌آل باشند.

۳. زبان SQL و اهمیت آن

SQL یا Structured Query Language زبان استاندارد برای ارتباط با دیتابیس‌های رابطه‌ای است. یادگیری SQL به معنای توانایی کار با هر سیستم مدیریت دیتابیس رابطه‌ای (RDBMS) است.

دستورات SQL به سه دسته اصلی تقسیم می‌شوند:

- DDL (Data Definition Language): برای تعریف ساختار داده مانند CREATE و ALTER.
- DML (Data Manipulation Language): برای کار با داده مانند SELECT، INSERT، UPDATE و DELETE.
- DCL (Data Control Language): برای کنترل دسترسی مانند GRANT و REVOKE.

SQL همچنین از مفاهیم پیشرفته‌ای مانند JOIN برای ترکیب داده‌های چند جدول، GROUP BY برای گروه‌بندی و توابع تجمعی مانند COUNT، SUM و AVG پشتیبانی می‌کند.

۴. PostgreSQL و ویژگی‌های آن

PostgreSQL یک سیستم مدیریت دیتابیس رابطه‌ای پیشرفته، رایگان و متن‌باز است که به دلیل پایداری کامل به استانداردهای SQL و قابلیت‌های مدرن خود شناخته می‌شود.

برخی از ویژگی‌های برجسته PostgreSQL:

- پشتیبانی از انواع داده پیشرفته مانند JSON و آرایه
- قابلیت Full-Text Search برای جستجوی متنی قدرتمند
- پشتیبانی از داده‌های جغرافیایی با PostGIS
- توابع پنجره‌ای (Window Functions) برای تحلیل داده
- امکان بازگرداندن داده پس از INSERT یا UPDATE با RETURNING

برای محیط توسعه و تست، PostgreSQL به راحتی با Docker راه‌اندازی می‌شود که فرآیند نصب و پیکربندی را بسیار ساده می‌کند.

۵. صفحه‌بندی، به‌روزرسانی و حذف داده‌ها

صفحه‌بندی (Pagination):

در API‌ها برای بازگرداندن حجم زیادی از داده، از صفحه‌بندی استفاده می‌شود. دو پارامتر کلیدی آن LIMIT و OFFSET هستند. LIMIT تعداد رکوردهای هر صفحه را مشخص می‌کند و OFFSET می‌گوید از کدام رکورد شروع کند. فرمول استاندارد به این صورت است: OFFSET برابر است با (شماره صفحه منهای یک) ضربدر تعداد رکوردهای هر صفحه.

به‌روزرسانی (UPDATE):

برای تغییر داده‌های موجود استفاده می‌شود. نکته حیاتی در استفاده از UPDATE، فراموش نکردن شرط WHERE است، در غیر این صورت تمام رکوردهای جدول تغییر می‌کنند.

حذف (DELETE):

برای پاک کردن داده‌ها استفاده می‌شود. مشابه UPDATE، استفاده از WHERE در DELETE نیز ضروری است تا از حذف ناخواسته تمام داده‌ها جلوگیری شود.

۶. JOIN و GROUP BY

JOIN:

عملیات JOIN امکان ترکیب داده‌ها از دو یا چند جدول را بر اساس یک ستون مشترک فراهم می‌کند. انواع اصلی JOIN عبارتند از:

INNER JOIN: فقط رکوردهایی را برمی‌گرداند که در هر دو جدول تطابق دارند.

LEFT JOIN: تمام رکوردهای جدول سمت چپ را با تطابق‌های جدول راست برمی‌گرداند.

RIGHT JOIN: عکس LEFT JOIN عمل می‌کند.

FULL OUTER JOIN: تمام رکوردهای هر دو جدول را برمی‌گرداند.

GROUP BY:

برای گروه‌بندی رکوردها بر اساس یک یا چند ستون و انجام محاسبات تجمعی روی هر گروه استفاده می‌شود. خروجی آن با توابعی مانند COUNT، SUM و AVG ترکیب می‌شود. برای فیلتر

کردن نتایج بعد از GROUP BY از HAVING استفاده می‌شود که مشابه WHERE برای گروه‌هاست.

۷. مسئله N+1 و راه‌حل‌های آن

مسئله N+1 یک مشکل رایج در کار با دیتابیس‌هاست که زمانی رخ می‌دهد که برنامه برای دریافت داده‌های مرتبط، به جای یک کوئری بهینه، چندین کوئری جداگانه ارسال می‌کند.

برای مثال، اگر ۱۰۰ کتاب داشته باشیم و برای نمایش نام نویسنده هر کتاب، یک کوئری جداگانه به جدول نویسندگان ارسال کنیم، مجموعاً ۱۰۱ کوئری انجام شده است (یک کوئری برای کتاب‌ها و ۱۰۰ کوئری برای نویسندگان).

راه‌حل‌های اصلی عبارتند از:

- استفاده از JOIN برای دریافت همه داده‌ها در یک کوئری
- Eager Loading در ORM ها که داده‌های مرتبط را از پیش بارگذاری می‌کند
- Batch Loading که داده‌های مرتبط را با یک کوئری IN جمع‌آوری می‌کند

۸. ORM و نقش آن

ORM مخفف Object-Relational Mapping، لایه‌ای نرم‌افزاری است که جداول دیتابیس را به کلاس‌های زبان برنامه‌نویسی (مانند پایتون) و رکوردها را به اشیاء نگاشت می‌دهد. به بیان ساده، ORM به برنامه‌نویس اجازه می‌دهد به جای نوشتن مستقیم SQL، با کد پایتون با دیتابیس کار کند.

مزایای اصلی ORM:

- کاهش نیاز به نوشتن SQL تکراری
- محافظت خودکار در برابر حملات SQL Injection
- مدیریت خودکار ارتباط با دیتابیس
- قابلیت تعویض دیتابیس بدون تغییر کد

محدودیت‌های ORM:

- کارایی کمتر در کوئری‌های بسیار پیچیده
- انتزاع بیش از حد که گاهی کنترل دقیق را از برنامه‌نویس می‌گیرد
- نیاز به یادگیری نحوه کار با ORM خاص

در اکوسیستم پایتون، SQLAlchemy محبوب‌ترین و قدرتمندترین ORM است.

۹. مفهوم کش (Cache)

کش یک حافظه موقت و سریع است که داده‌های پرکاربرد را در خود ذخیره می‌کند. هدف اصلی آن کاهش زمان پاسخ و کاهش بار روی منبع اصلی داده (مانند دیتابیس) است.

انواع کش در توسعه وب:

- In-Memory Cache: داده‌ها در حافظه همان پروسه اپلیکیشن ذخیره می‌شوند. سریع است ولی بین نمونه‌های مختلف اپلیکیشن به اشتراک گذاشته نمی‌شود.
- سرور کش (مانند Redis): یک سرویس جداگانه که داده‌ها را در حافظه نگه می‌دارد و چندین نمونه اپلیکیشن می‌توانند از آن استفاده کنند.
- HTTP Caching: مرورگر با استفاده از هدرهایی مانند Cache-Control و ETag می‌تواند پاسخ‌ها را کش کند.
- CDN (شبکه توزیع محتوا): محتوای استاتیک در سرورهایی در نقاط مختلف جهان توزیع می‌شود تا کاربر از نزدیک‌ترین نقطه خدمات بگیرد.
- Full-Cache: کش کردن کل خروجی یک صفحه یا API، مناسب برای محتوایی که به ندرت تغییر می‌کند.

۱۰. Redis و کاربردهای آن

Redis یک دیتابیس کلید-مقدار در حافظه است که به دلیل سرعت بسیار بالا (پاسخ در حد میکروثانیه) شناخته می‌شود. برخلاف کش‌های ساده In-Memory، Redis قابلیت‌های پیشرفته‌ای دارد.

ویژگی‌های برجسته Redis:

- انواع داده متنوع: علاوه بر رشته ساده، از لیست، مجموعه، مجموعه مرتب و هش پشتیبانی می‌کند.
- تعیین زمان انقضا (TTL): می‌توان برای هر کلید مدت اعتبار تعیین کرد.
- ماندگاری (Persistence): قابلیت ذخیره داده‌ها روی دیسک برای بازیابی پس از ریستارت.
- انتشار/اشتراک (Pub/Sub): برای پیاده‌سازی سیستم‌های پیام‌رسان.
- مقیاس‌پذیری: قابلیت اجرا به صورت کلاستر.

مقایسه Redis با کش In-Memory ساده پایتون:

Redis بین چندین پروسه و سرور به اشتراک گذاشته می‌شود، اما دیکشنری پایتون فقط در همان پروسه قابل دسترسی است. Redis داده را پس از ریستارت حفظ می‌کند، اما دیکشنری پایتون با پایان برنامه از بین می‌رود. Redis برای محیط تولید مناسب است، اما کش ساده پایتون برای توسعه و تست کافی است.

۱۱. Connection Pooling و اهمیت آن

ایجاد یک اتصال جدید به دیتابیس عملیاتی زمان‌بر و پرهزینه است. Connection Pooling راه‌حلی برای مدیریت کارآمد این اتصالات است.

روش کار Connection Pool:

به جای باز و بسته کردن مکرر اتصال، مجموعه‌ای از اتصالات آماده (Pool) از پیش ساخته می‌شود. هنگامی که برنامه نیاز به دیتابیس دارد، یک اتصال از این مجموعه قرض می‌گیرد و پس از اتمام کار، آن را به مجموعه برمی‌گرداند.

مزایای اصلی:

- کاهش چشمگیر زمان انتظار برای برقراری اتصال
- استفاده بهینه از منابع سرور دیتابیس
- مدیریت خودکار اتصالات قطع شده و ایجاد مجدد آن‌ها
- جلوگیری از ایجاد تعداد بی‌رویه اتصال هم‌زمان

در پایتون، کتابخانه psycopg2 که برای ارتباط با PostgreSQL استفاده می‌شود، این قابلیت را به صورت داخلی فراهم کرده است.

۱۲. جمع‌بندی نهایی

- در این هفته، مباحث پایه‌ای و حیاتی برای توسعه بک‌اند را فرا گرفتیم:
- از ساختار دیتابیس‌های رابطه‌ای و زبان SQL شروع کردیم
- با PostgreSQL و نحوه راه‌اندازی آن با Docker آشنا شدیم
- مفاهیم پیشرفته SQL مانند JOIN و GROUP BY را آموختیم
- مسئله N+1 و راه‌حل‌های آن را درک کردیم
- مفهوم ORM و مزایا و معایب آن را بررسی کردیم
- کش و انواع آن را شناختیم و Redis را به عنوان یک راه‌حل Production-ready بررسی کردیم
- Connection Pooling و اهمیت آن در برنامه‌های واقعی را فهمیدیم